

TaskFlow

Toutes les requêtes Postman

Guide complet de l'API locale - sujet A : gestion de tâches personnelles

Démarrer et importer

L'API et la base de données doivent tourner. Dans le premier terminal, démarrer MongoDB avec `npm run db:local`. Dans le second, démarrer Express et Vite avec `npm run dev`. Le site visuel se consulte dans un navigateur à `http://localhost:5173`; Postman sert à interroger l'API.

API	<code>http://localhost:3000</code>
Interface web	<code>http://localhost:5173</code>
Swagger	<code>http://localhost:3000/api/docs/</code>
OpenAPI	<code>http://localhost:3000/api/openapi.json</code>

Dans Postman, importer `TaskFlow.postman_collection.json`. La collection crée automatiquement des adresses de test distinctes, conserve `tokenA`, `tokenB` et `taskId`, puis ajoute le jeton Bearer aux opérations privées. Exécuter les dossiers numérotés dans l'ordre. Les jetons expirent après une heure : relancer la requête de connexion correspondante si nécessaire.

Pour saisir une requête manuellement, créer un environnement avec `baseUrl = http://localhost:3000`, puis remplir `tokenA`, `tokenB` et `taskId` avec les valeurs reçues. Pour les requêtes JSON, choisir **Body** > **raw** > **JSON**. Pour une route privée, choisir **Authorization** > **Bearer Token** et fournir le jeton du compte visé.

200	Lecture, connexion ou modification réussie.
201	Création réussie.
204	Suppression ou changement de mot de passe, sans corps.
400	Entrée, identifiant ou valeur invalide.
401	Authentification absente ou refusée.
404	Objet absent ou appartenant à un autre compte.
409	Adresse email déjà utilisée.

Table des matières

1	1 - Vérifier les serveurs	2
2	2 - Compte A et authentification	3
3	3 - CRUD des tâches avec A	4
4	4 - Confidentialité entre A et B	6
5	5 - Mot de passe et absence de jeton	8

1 1 - Vérifier les serveurs

Contrôles publics : santé, proxy Vite, OpenAPI, Swagger et route inconnue.

API - Santé

GET

```
GET http://localhost:3000/api/health
```

Réponse attendue : HTTP 200 Corps : "status" : "ok".

API - Spécification OpenAPI

GET

```
GET http://localhost:3000/api/openapi.json
```

Réponse attendue : HTTP 200

API - Interface Swagger

GET

```
GET http://localhost:3000/api/docs/
```

Réponse attendue : HTTP 200

API - Réponse 404 sur route inconnue

GET

```
GET http://localhost:3000/api/does-not-exist
```

Réponse attendue : HTTP 404 Erreur : NOT_FOUND.

2 2 - Compte A et authentification

Créer A puis récupérer son JWT. Les deux routes d'authentification sont publiques.

A - Créer un compte

POST

Première exécution : crée A avec une adresse de test. La collection enregistre son tokenA. Si le compte existe déjà, utiliser A - Se connecter.

```
POST http://localhost:3000/api/auth/register
Content-Type: application/json

Body (raw / JSON):
{
  "email": "{{emailA}}",
  "password": "{{passwordA}}"
}
```

Réponse attendue : HTTP 201 Réponse : user.id, user.email et token. Aucun mot de passe ni hash n'est renvoyé.

A - Email déjà utilisé (409)

POST

Après l'inscription A, réutiliser le même email. Le serveur refuse la deuxième inscription.

```
POST http://localhost:3000/api/auth/register
Content-Type: application/json

Body (raw / JSON):
{
  "email": "{{emailA}}",
  "password": "{{passwordA}}"
}
```

Réponse attendue : HTTP 409 Erreur : EMAIL_ALREADY_USED.

A - Se connecter

POST

Récupère un nouveau tokenA après la création de A. Exécuter après A - Créer un compte ou plus tard pour renouveler le jeton.

```
POST http://localhost:3000/api/auth/login
Content-Type: application/json

Body (raw / JSON):
{
  "email": "{{emailA}}",
  "password": "{{passwordA}}"
}
```

Réponse attendue : HTTP 200

3 3 - CRUD des tâches avec A

La création enregistre automatiquement taskId, utilisé par le détail, la modification et la suppression.

A - Lister mes tâches

GET

```
GET http://localhost:3000/api/tasks
Authorization: Bearer {{tokenA}}
```

Réponse attendue : HTTP 200 Réponse : "items" :[...]; une liste vide vaut "items" :[].

A - Créer une tâche

POST

```
POST http://localhost:3000/api/tasks
Authorization: Bearer {{tokenA}}
Content-Type: application/json

Body (raw / JSON):
{
  "title": "Préparer la démonstration",
  "status": "todo",
  "description": "Tester les requêtes dans Postman.",
  "dueDate": "2026-10-20"
}
```

Réponse attendue : HTTP 201 La réponse contient la tâche créée avec son identifiant public id.

A - Voir la tâche

GET

```
GET http://localhost:3000/api/tasks/{{taskId}}
Authorization: Bearer {{tokenA}}
```

Réponse attendue : HTTP 200

A - Modifier partiellement (PATCH)

PATCH

```
PATCH http://localhost:3000/api/tasks/{{taskId}}
Authorization: Bearer {{tokenA}}
Content-Type: application/json

Body (raw / JSON):
{
  "status": "doing"
}
```

Réponse attendue : HTTP 200

A - Rejeter un statut invalide

POST

```
POST http://localhost:3000/api/tasks
Authorization: Bearer {{tokenA}}
Content-Type: application/json

Body (raw / JSON):
{
  "title": "Statut refusé",
  "status": "archived"
}
```

Réponse attendue : HTTP 400 Erreur : INVALID_INPUT.

A - Rejeter un champ ownerId falsifié

POST

```
POST http://localhost:3000/api/tasks
Authorization: Bearer {{tokenA}}
Content-Type: application/json

Body (raw / JSON):
{
  "title": "Propriétaire falsifié",
  "status": "todo",
  "ownerId": "507f1f77bcf86cd799439011"
}
```

Réponse attendue : HTTP 400 Erreur : INVALID_INPUT.

A - Identifiant malformé

GET

```
GET http://localhost:3000/api/tasks/identifiant-invalide
Authorization: Bearer {{tokenA}}
```

Réponse attendue : HTTP 400 Erreur : INVALID_INPUT.

A - Supprimer la tâche

DELETE

La suppression réussie ne renvoie aucun corps.

```
DELETE http://localhost:3000/api/tasks/{{taskId}}
Authorization: Bearer {{tokenA}}
(aucun corps)
```

Réponse attendue : HTTP 204 Succès sans corps de réponse.

A - Vérifier que la tâche supprimée est absente

GET

```
GET http://localhost:3000/api/tasks/{{taskId}}
Authorization: Bearer {{tokenA}}
```

Réponse attendue : HTTP 404 Erreur : NOT_FOUND.

4 4 - Confidentialité entre A et B

B crée son propre jeton et vérifie qu'il ne voit ni ne modifie les objets de A.

B - Créer un compte

POST

Crée B avec une adresse différente et enregistre son tokenB.

```
POST http://localhost:3000/api/auth/register
Content-Type: application/json

Body (raw / JSON):
{
  "email": "{{emailB}}",
  "password": "{{passwordB}}"
}
```

Réponse attendue : HTTP 201

B - Se connecter

POST

```
POST http://localhost:3000/api/auth/login
Content-Type: application/json

Body (raw / JSON):
{
  "email": "{{emailB}}",
  "password": "{{passwordB}}"
}
```

Réponse attendue : HTTP 200

B - Liste privée (vide au départ)

GET

Exécuter avant de créer une tâche pour B. Aucune tâche de A ne doit apparaître.

```
GET http://localhost:3000/api/tasks
Authorization: Bearer {{tokenA}}
```

Réponse attendue : HTTP 200 Réponse : "items" :[] si B n'a pas encore créé de tâche.

A - Créer une tâche privée

POST

```
POST http://localhost:3000/api/tasks
Authorization: Bearer {{tokenA}}
Content-Type: application/json

Body (raw / JSON):
{
  "title": "Tâche privée de A",
  "status": "todo"
}
```

Réponse attendue : HTTP 201 La réponse contient la tâche créée avec son identifiant public id.

B - Lecture de la tâche A refusée**GET**

```
GET http://localhost:3000/api/tasks/{{taskId}}
Authorization: Bearer {{tokenB}}
```

Réponse attendue : HTTP 404 Erreur : NOT_FOUND.

B - Modification de la tâche A refusée**PATCH**

```
PATCH http://localhost:3000/api/tasks/{{taskId}}
Authorization: Bearer {{tokenB}}
Content-Type: application/json

Body (raw / JSON):
{
  "status": "done"
}
```

Réponse attendue : HTTP 404 Erreur : NOT_FOUND.

B - Suppression de la tâche A refusée**DELETE**

```
DELETE http://localhost:3000/api/tasks/{{taskId}}
Authorization: Bearer {{tokenB}}
(aucun corps)
```

Réponse attendue : HTTP 404 Erreur : NOT_FOUND.

A - Sa tâche est toujours là**GET**

```
GET http://localhost:3000/api/tasks/{{taskId}}
Authorization: Bearer {{tokenA}}
```

Réponse attendue : HTTP 200

A - Nettoyage : supprimer sa tâche**DELETE**

```
DELETE http://localhost:3000/api/tasks/{{taskId}}
Authorization: Bearer {{tokenA}}
(aucun corps)
```

Réponse attendue : HTTP 204 Succès sans corps de réponse.

5 5 - Mot de passe et absence de jeton

Modifier le mot de passe de A ou vérifier le refus d'une route privée sans authentification.

A - Connexion refusée (mot de passe incorrect)

POST

```
POST http://localhost:3000/api/auth/login
Content-Type: application/json

Body (raw / JSON):
{
  "email": "{{emailA}}",
  "password": "mauvais-mot-de-passe"
}
```

Réponse attendue : HTTP 401 Erreur : UNAUTHORIZED.

A - Modifier le mot de passe

PATCH

Remplace `passwordA` par `newPassword` dans les variables après succès afin de pouvoir ensuite se reconnecter.

```
PATCH http://localhost:3000/api/auth/password
Authorization: Bearer {{tokenA}}
Content-Type: application/json

Body (raw / JSON):
{
  "currentPassword": "{{passwordA}}",
  "newPassword": "{{newPassword}}"
}
```

Réponse attendue : HTTP 204 Succès sans corps de réponse.

Tâches sans connexion : refusées

GET

```
GET http://localhost:3000/api/tasks
```

Réponse attendue : HTTP 401 Erreur : UNAUTHORIZED.

A - Inscription invalide (email)

POST

```
POST http://localhost:3000/api/auth/register
Content-Type: application/json

Body (raw / JSON):
{
  "email": "pas-une-adresse",
  "password": "TaskflowDemo123!"
}
```

Réponse attendue : HTTP 400 Erreur : INVALID_INPUT.

Notes d'utilisation

- Les routes des tâches exigent un JWT : `tokenA` pour A et `tokenB` pour B. Les deux comptes ont des tâches distinctes.
- Le champ `taskId` vient de la réponse à `POST /api/tasks`. Si la variable est vide, exécuter d'abord la création.
- Dans le scénario A/B, B doit recevoir 404 pour GET, PATCH et DELETE sur une tâche qui appartient à A. Refaire un GET avec tokenA confirme que l'objet de A existe toujours.
- Pour réutiliser la collection, relancer les requêtes de connexion si les jetons ont expiré. Si une adresse de test est déjà utilisée, choisir une autre adresse ou exécuter la connexion plutôt que l'inscription.
- Les comptes créés demeurent dans la base locale. Supprimer les tâches de test avec DELETE après la démonstration. Le projet ne propose pas de suppression de compte.
- Les mots de passe de la collection sont réservés à une démonstration locale. Ne pas les réutiliser pour un compte personnel.

Guide établi à partir du contrat API TaskFlow du livret étudiant V2 et du fichier OpenAPI livré avec le projet. Les exemples utilisent exclusivement des données fictives.